

[deprecated] Mocha

Wasaby предоставляет возможность покрывать код интерфейсных модулей юнит-тестами, которые могут запускаться как в окружении Node.js, так и в браузере, с помощью фреймворка [Mocha \(Мокка\)](#).

 Данный фреймворк устарел. Рекомендуется использовать [Jest](#).

Принципы организации

1. Для хранения юнит-тестов интерфейсного модуля нужно создать отдельный модуль (папка содержащая одноимённый файл с расширением .s3mod). Обязательно заполнить список его зависимостей, как минимум там должен быть указан тестируемый модуль, и добавить тег `<unit_test/>`. Имя модуля задается в виде `<ИмяТестируемогоМодуля>Unit`, например, для модуля *Controls* модуль с тестами должен называться *ControlsUnit*.

Пример s3mod-файла для модуля с юнит-тестами:

```
1 <?xml version="1.0" encoding="UTF-8" ?>
2 <ui_module id="1234" name="<ИмяТестируемогоМодуля>Unit"
3     responsible="Иванов И.И." version="1.00">
4     <description/>
5     <depends>
6         <module id="0000" name="<ИмяТестируемогоМодуля>" />
7     </depends>
8     <load_after/>
9     <unit_test/>
10    <parameters/>
11    <resources/>
12    <preload/>
13 </ui_module>
```

2. Пакеты тестов должны иметь имена файлов, заканчивающиеся на `.test.ts`. Для каждого тестируемого файла (модуля) нужно создать одноименный пакет тестов.
3. При тестировании класса каждое свойство или метод тестируются внутри отдельного `score`, заданного через вложенный вызов `describe()`.

Примеры кода

Для тестирования класса `Types/_entity/CancelablePromise` нужно создать пакет тестов `TypesUnit/_entity/CancelablePromise.test.ts` со следующим содержанием:

```
1 import {assert} from 'chai';
2 import CancelablePromise from 'Types/_entity/CancelablePromise';
3
4 describe('Types/_entity/CancelablePromise', () => {
5     describe('.promise', () => {
6         it('should return instance of original Promise', () => {
7             const origin = new Promise((resolve) => resolve());
8             const instance = new CancelablePromise(origin);
9             assert.strictEqual(instance.promise, origin);
10         });
11     });
12 }
```

```
13     // Далее тестируем другие свойства и методы класса
    CancelablePromise
14 });
```

Для подключения библиотеки Sinon используйте следующий **import**:

```
1 import * as sinon from 'sinon';
2
3 // ...
```

Запуск тестов

Для запуска тестов используйте сценарий **runUnitTests**:

```
npx wasaby-cli --tasks=runUnitTests
```

По умолчанию результат выполнения тестов складывается в папку **artifacts**. Для вывода результатов в консоль используйте аргумент **report**:

```
npx wasaby-cli --tasks=runUnitTests --report=console
```

По умолчанию тесты запускаются в Node.js и браузере. Если вы хотите запускать тесты только в одном окружении, необходимо добавить флаг **node** для Node.js и **browser** для браузера.

Только в Node.js:

```
npx wasaby-cli --tasks=runUnitTests --node
```

Только в браузере вы не увидите знакомого браузерного окна, тесты пройдут незаметно для вас и отладиться вы не сможете:

```
npx wasaby-cli --tasks=runUnitTests --browser
```

Для запуска тестов в браузере с возможностью отладки запустите http-сервер командой:

```
npx wasaby-cli --tasks=runUnitTests --server
```

В консоли будет написано, на каком порту он запустился (свободный порт выбирается автоматически). Для выполнения юнит-тестов в браузере откройте URL <http://localhost:port>, заменив номер порта на указанный в консоли.

Полный список аргументов, используемых для запуска юнит-тестов:

- **tasks=runUnitTests** — запустить сценарий юнит-тестирования.
- **rep=[name]** — название тестируемого пакета (соответствует *name* в *package.json*).
- **rc=rc-version** — запустить юнит-тесты только по измененным модулям. Если нужно запустить все тесты, можно не указывать.
- **only** — запускать тесты только по одному репозиторию, указанному в *rep*.

Отладка юнит-тестов

Подробнее про отладку тестов под Node.js читайте в отдельной [статье](#).

Автоматизация

Для удобства вы можете добавить указанные выше команды в раздел **scripts** файла **package.json**.

Типовой набор выглядит так:

```
1  "scripts": {
2    "build": "npm run build:deps && npm run build:app",
3    "build:deps": "wasaby-cli --tasks=initStore",
4    "build:app": "wasaby-cli --tasks=build",
5    "start": "wasaby-cli --tasks=runUnitTests --server",
6    "test": "wasaby-cli %NODE_DEBUG_OPTION% --tasks=runUnitTests --node
--report=console"
7  }
```

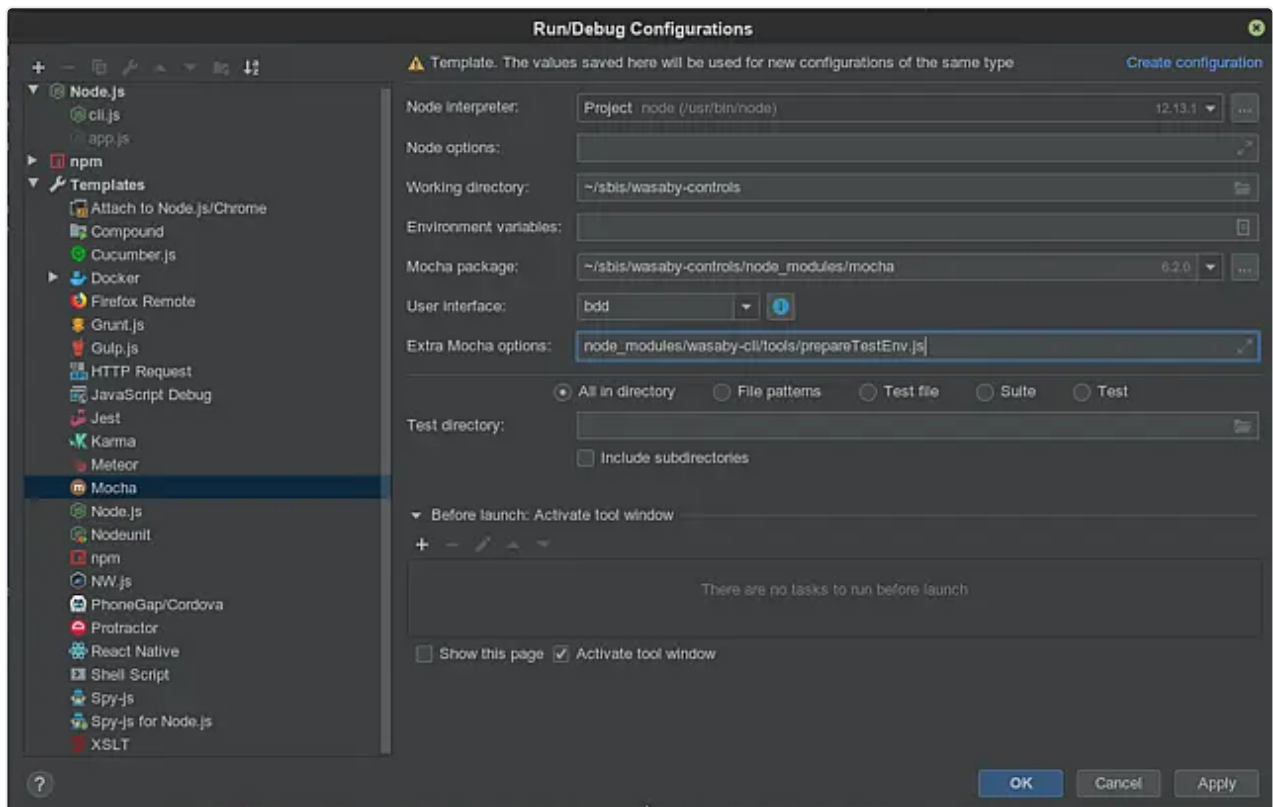
Многие IDE предоставляют возможность запускать эти скрипты через визуальный интерфейс.

Интеграция с WebStorm

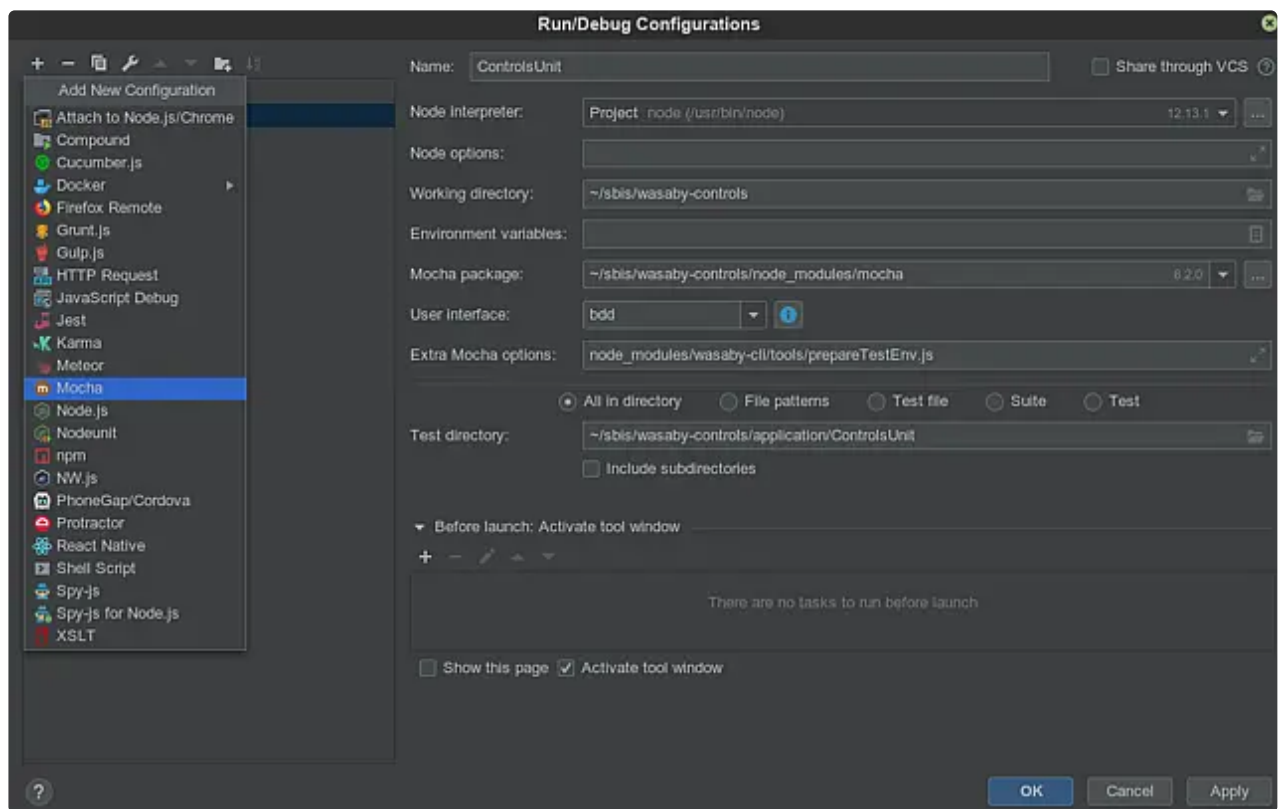
Помимо запуска тестов с командной строки можно интегрировать их в интерфейс IDE WebStorm, а также запускать тесты по одному прямо из редактора.

Инструкция по интеграции:

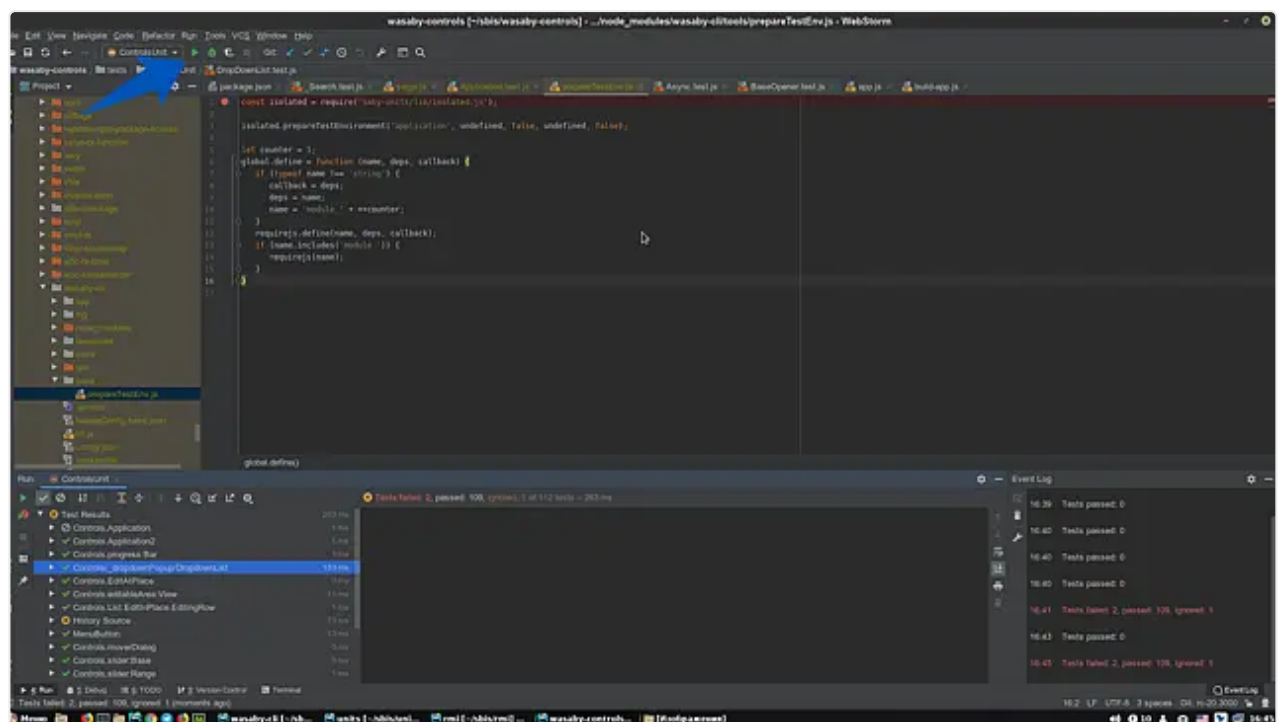
1. Открыть **Run/Debug Configurations**, развернуть в левом меню **templates** и выбрать там **Mocha**.
2. Заполнить пункты:
 - Working directory: корень репозитория
 - Mocha package: node_modules/mocha
 - User interface: bdd
 - Extra Mocha options: node_modules/wasaby-cli/tools/prepareTestEnv.js



3. Добавить конфигурацию Mocha, и выбрать Test directory: application/ControlsUnit.



4. Нажать кнопку запуска:



5. Чтобы запустить только один тест, нажмите на зеленую точку около его описания:

